

CS & IT ENGINEERING



Data Structure & Programming

Linked Lists

Lecture No.- 04



By- Satya sir

Recap of Previous Lecture



DPP - Linked List

Q.8 $ptx = (\text{NODE} *) \text{malloc}(\text{sizeof}(\text{NODE}))$ ANS : A

Last class

Doubly Linked List :

- Node Structure

- DLL - Insertion

Beginning ✓

End ✓

Position



Topics to be Covered



- DLL
 - Insertion at middle
- Deletion
 - First Node
 - Last Node
 - middle Node
- Time Complexities

- Circular Linked List
 - operations.



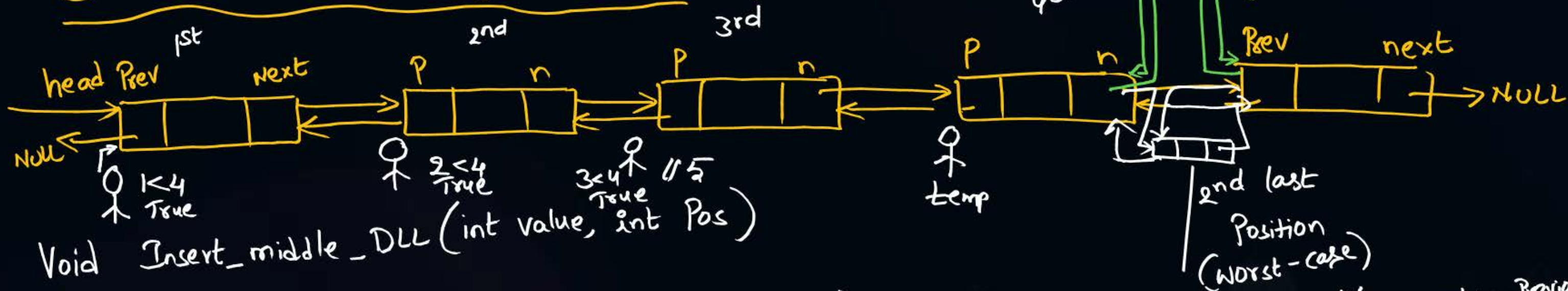
Topic : Doubly Linked List-2



Time Complexity : $O(n)$

DLL - Insertion At given Position

// temp reaching till given Position
 while (Count < Pos) { temp = temp->next; Count++; }
 { new->prev = temp->prev;
 new->prev->next = new;
 new->next = temp;
 temp->prev = new; }



Void Insert_middle_DLL (int value, int Pos)

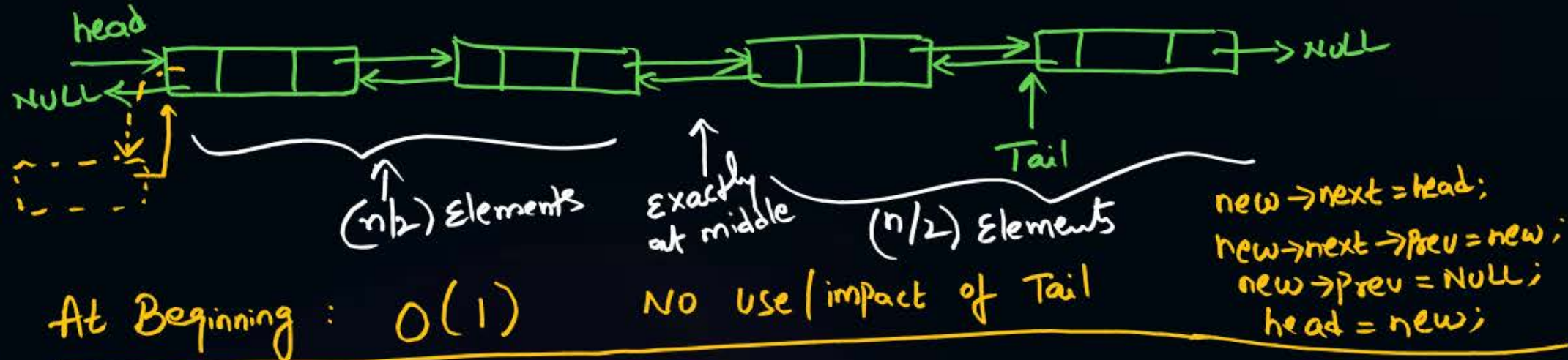
```
{
    struct DLL *new;
    new = (struct *) malloc (Size of (struct DLL));
    new->data = value;
    if (head == NULL) // Empty list
    {
        head = new;
        new->prev = NULL;
        new->next = NULL;
        return;
    }
    else if (head->next == NULL) // one Node
    {
        new->next = head;
        new->next->prev = new;
        new->prev = NULL;
        head = new;
    }
    else // temp reaching Previous Node of given pos
    {
        struct DLL *temp = head;
        int Count = 1;
        while (Count < Pos-1)
        {
            temp = temp->next;
            Count++;
        }
        new->next = temp->next;
        new->next->prev = new;
        temp->next = new;
        new->prev = temp;
    }
}
```




Topic : Doubly Linked List-2



Insertion into DLL with Tail Pointer



At End : Without Tail $O(n)$ With Tail

Code for insertion at end with Tail:

```
Tail->next = new;
new->prev = Tail;
new->next = NULL;
Tail = new;
```

$O(1)$

At middle

Without Tail	With Tail
Worst case Position will be 2nd last Position $O(n)$	Worst Case Position will be $(n/2)^{th}$ [Exactly middle] Position $\Rightarrow$ Time Complexity: $O(n)$



Topic : Doubly Linked List-2

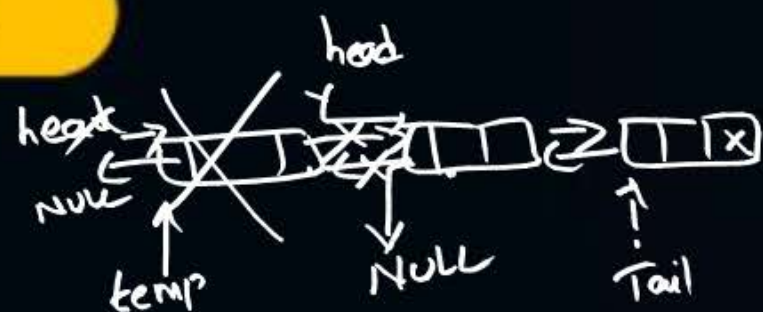
Deletion from DLL

First Node [Without / With Tail]

```

Void Delete_first_DLL( )
{
    if (head == NULL)
    {
        Pf("Empty list");
    }
    else if (head->next == NULL)
    {
        Struct DLL *temp = head;
        head = NULL;
        free(temp);
    }
}
    
```

Time complexity: $O(1)$



```

else
{
    Struct DLL *temp = head;
    head = head->next;
    temp->next->prev = NULL;
    temp->next = NULL;
    free(temp);
}
    
```

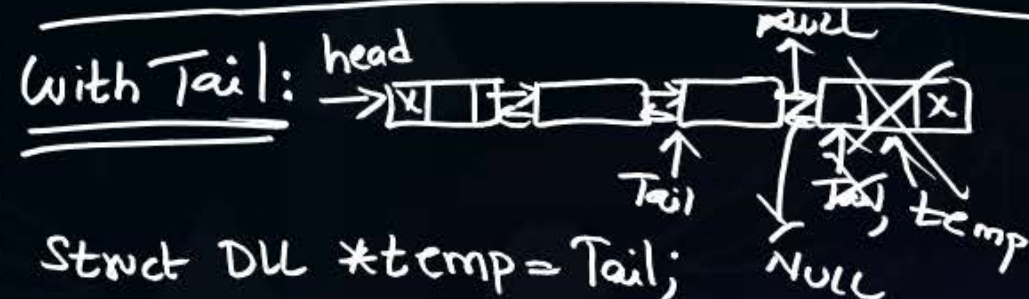
Last Node

Without Tail

```

Struct DLL *temp = head;
while (temp->next != NULL)
    temp = temp->next;
temp->prev->next = NULL;
temp->prev = NULL;
free(temp);
    
```

T.C.
 $O(n)$



```

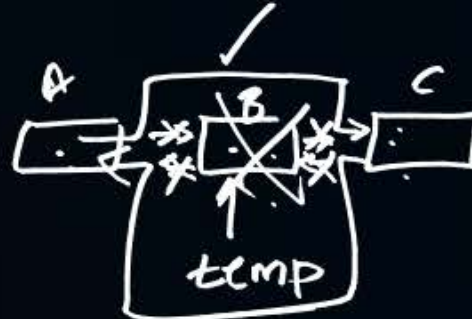
Struct DLL *temp = Tail;
Tail = Tail->prev;
Tail->next = NULL;
Temp->prev = NULL;
free(temp);
    
```

Time complexity
 $O(1)$





Topic : Doubly Linked List-2



Deletion of middle Node

Without Tail : 2nd last Position == Worst case
Time Complexity : $O(n)$

With Tail : Exact middle $((n/2)^{th})$ Position
= Worst case.
Time Complexity : $O(n)$

Search operation:

Best: $O(1)$, Avg, worst: $O(n)$

```

Struct DL *temp = head; (OR) Struct DL *temp = Tail;
    count = 1;                count = n;
while (count < Pos)          while (count > Pos)
{
    temp = temp->next;        { temp = temp->prev;
    count++;                  }
}                             count--;

```

temp->prev->next = temp->next;

temp->next->prev = temp->prev;

temp->prev = NULL;

temp->next = NULL;

free(temp);

Time Complexity : $O(n)$



Circular Linked List

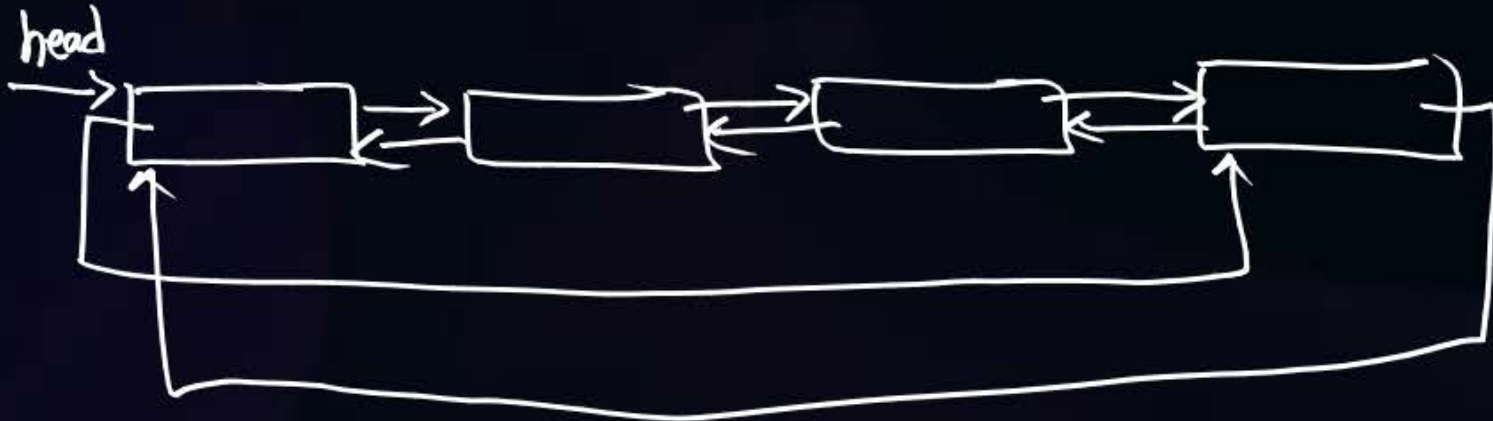
In SLL : Each node except last node connects to next node.

In Circular SLL : Every node connects to next node.



In DLL : Each node connects to both Prev and next nodes except First and last Nodes.

In Circular DLL : Every node connects to 2 other nodes next node and Previous node.





Topic : Doubly Linked List-2



Node structure will be same as SLL, DLL.

Struct CLL

```
{
    data;
    struct CLL *next;
} *head = NULL;
```

Struct CDLL

```
{
    struct CDLL *prev;
    data;
    struct CDLL *next;
} *head = NULL;
```

Insertion At Beginning / Deletion of first Node

CSLL/CLL: if Empty list : head = new; new → next = head;



without Tail : $O(n)$

with Tail : $O(1)$



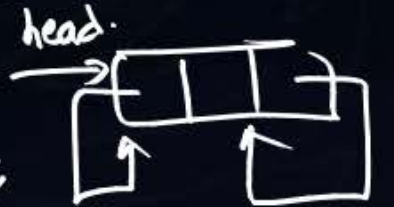
In CLL
End of list / last node check

```
if (temp->next == head)
    // last Node.
```



CDLL

if Empty list: head = new;
new → prev = head;
new → next = head;



without Tail /

with Tail

: $O(1)$



Topic : Doubly Linked List-2



Deletion of last Node

Without Tail : CSLL : $O(n)$
CDLL : $O(1)$

With Tail : CSLL : $O(n)$
CDLL : $O(1)$

Deletion of middle node

$O(n)$ in all Scenarios / Case Studies



2 mins Summary



- DLL operations

- CLL operations.



THANK - YOU